DECA Lab Logbook — Spring Term

Lab 1 | Task 3: Data Flow from Register File to ALU

Task 3: Data Flow from Register File to ALU

Objectives

- How to use waveform viewers together with truth tables to understand hardware logic
- How fields a, b, and c control register reads and writes in more detail, and how their roles differ for different instruction types
- How data flows differently for different instructions (MOV and ADD) and for different forms (register vs. immediate)
- The purpose of the op2sel control signal in the context of immediate (#IMM) operands
- The purpose of the ad1sel control signal in the context of write addresses for arithmetic operations with three operands
- How data flows between the ALU and the register file

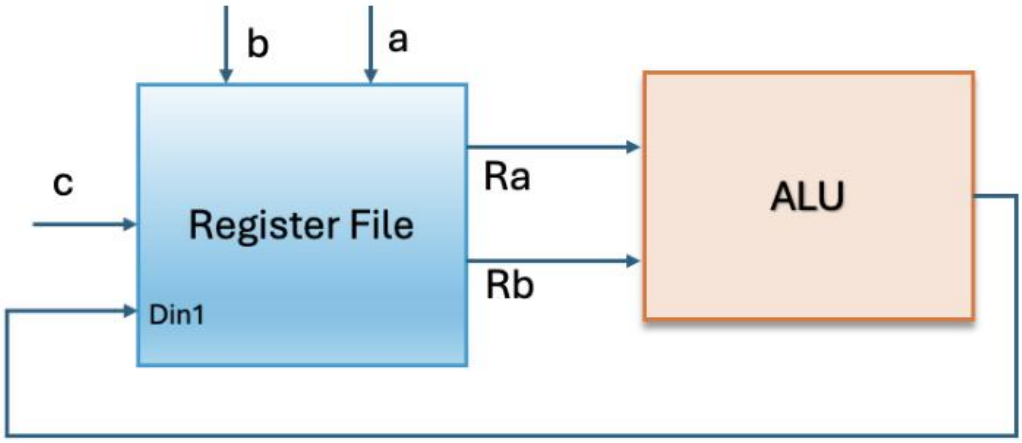


Figure 3: EEP1 datapath simplified

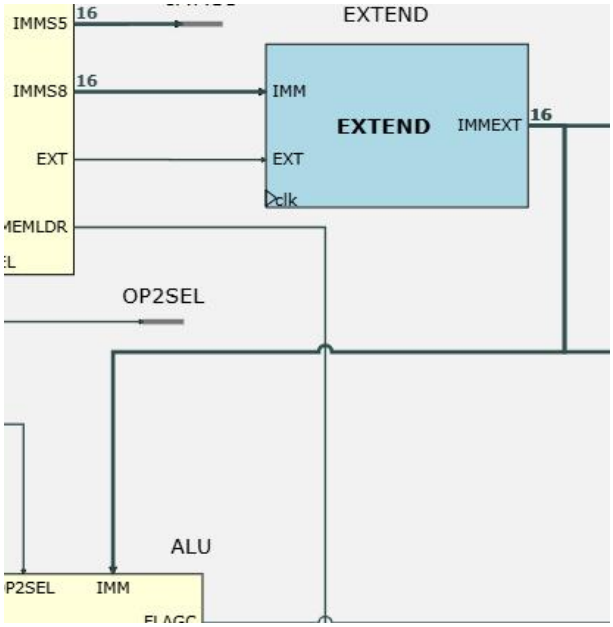
```
MOV R0, #3
MOV R1, R0
ADD R1, #1
ADD R3, R0, R1
```

1: For each register file data output port (for example DOUT2, DOUT3): • state which address field in the instruction word (a, b, or c) controls which register is read on that port; • state which ALU input port this register file output is connected to.

- DOUT2 is addressed by AD2, which receives field a (bits[11:9]) from dpdecode port A. DOUT2 connects to ALU input RA.
- DOUT3 is addressed by AD3, which receives field b (bits[7:5]) from dpdecode port B. DOUT3 connects to ALU input RB.

2: Immediate value into the ALU. Name the ALU input port through which immediate values enter (this is the same port you used in Section 2) and confirm how it is connected to dpdecode.

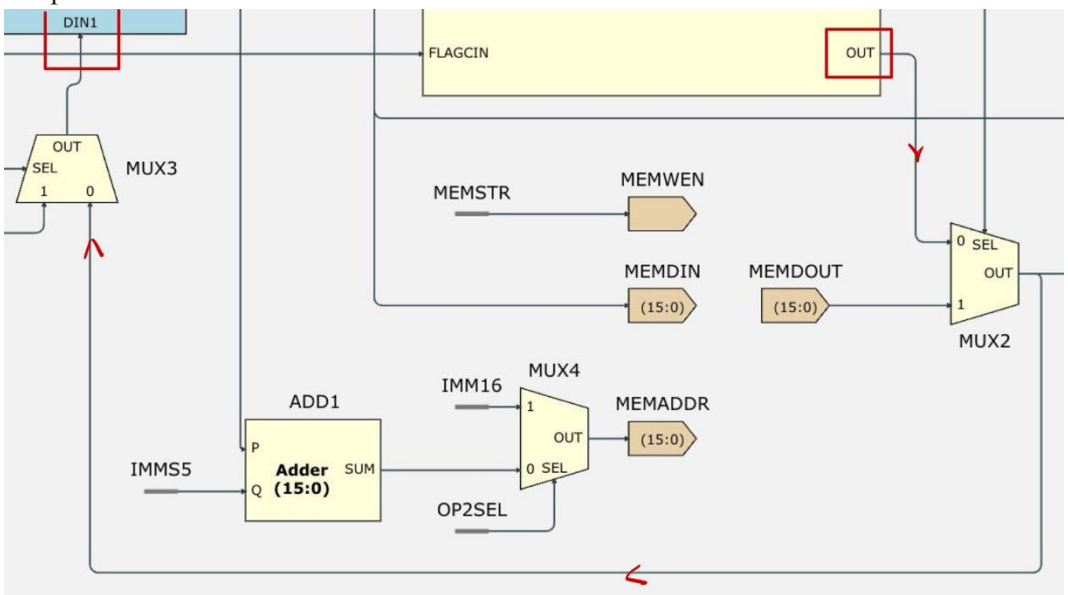
Immediate values enter the ALU through port IMM. The sign-extended 16-bit immediate (IMMS8 from dpdecode) passes through the EXTEND block as IMMEXT and connects directly to the ALU's IMM input. This path is active when OP2SEL = 1.



3: ALU output back to register file. Name the output port of the ALU that carries the result of operations. Explain how this output is routed back to the register file so that the result can be stored in a register. In particular, identify: • which register file data input port it connects to; • how the register write address is selected in these instructions.

The ALU result leaves through port OUT. The write-back path is:

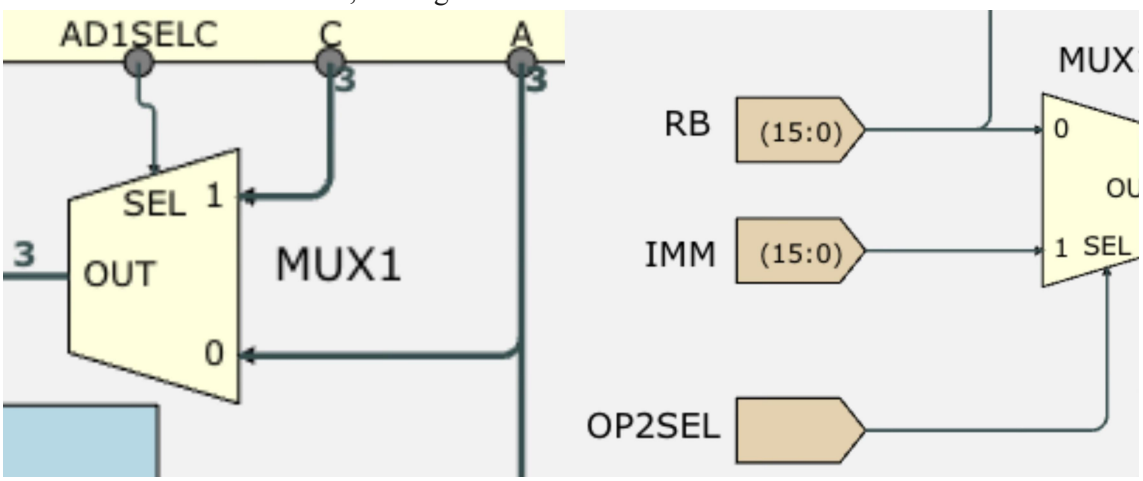
- ALU OUT → MUX2 (select: MEMLDR). MEMLDR = 0 passes the ALU result; MEMLDR = 1 selects memory data (MEMDOUT) for load instructions.
- MUX2 OUT → MUX3 (select: PCWRITE). PCWRITE = 0 forwards the result to register file input DIN1. The write address AD1 is derived from the instruction field via AD1SELC.



4: Control signals op2sel and ad1sel. The signals op2sel and ad1sel are 1-bit control signals from dpdecode. They control how data flows into the ALU and register file. In this question you will determine their purpose and for which instructions they are 0 or 1. One useful method is: 1: Construct a 4-row truth table… 2: Using your truth table and the ALU Instructions : detail sheet, explain what conditions cause each signal to be 1. 3: Trace op2sel and ad1sel to the multiplexers (muxes) they control.

A	B	C	IMM	OP2SEL	AD1SEL	REGFILE.Din1	REGFILE.AD1
a	b	c	n	0	0	ALU.OUT(A, B)	a
a	b	c	n	0	1	ALU.OUT(A, B)	c
a	b	c	n	1	0	ALU.OUT(A, IMM)	a
a	b	c	n	1	1	ALU.OUT(A, IMM)	c

- OP2SEL = INS(8). Selects the ALU second operand: 0 → RB (register Rb); 1 → IMM (sign-extended immediate). Controls MUX1 between RB and IMM.
- AD1SEL (AD1SELC). Selects the write address: 0 → field a (Ra); 1 → field c (Rc). Three-operand arithmetic instructions (ADD Rc, Ra, Rb) set AD1SELC = 1 so that Rc is written. MOV uses AD1SELC = 0, writing to Ra.

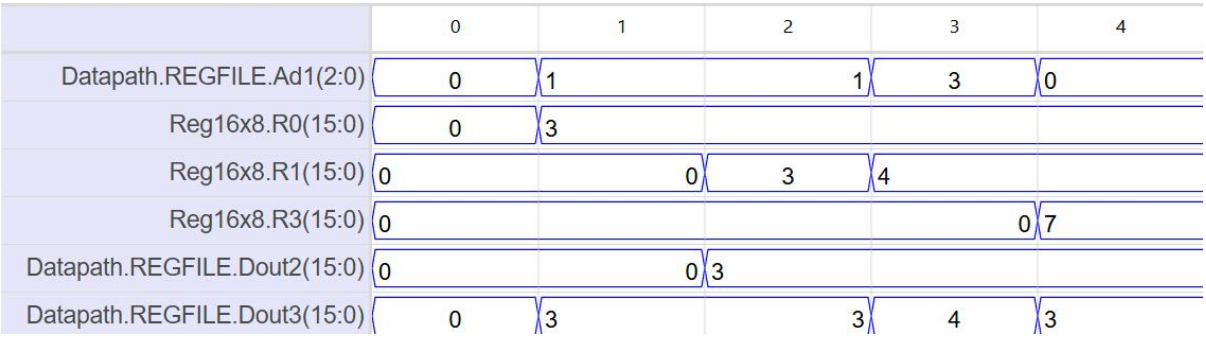


Extension

1: When executing a MOV Ra, #IMM instruction, how does the register read port of the register file behave? Does it still output register data? Explain what happens to this data and which hardware arrangement ensures that the immediate value, rather than the register output, is written into the destination register.

The read ports DOUT2 and DOUT3 continue to output register values, since they are purely combinational. However, with OP2SEL = 1, the MUX selects the sign-extended immediate rather than RB before the ALU. The register values on the read ports are therefore ignored; only the immediate value reaches DIN1 and is written to Ra.

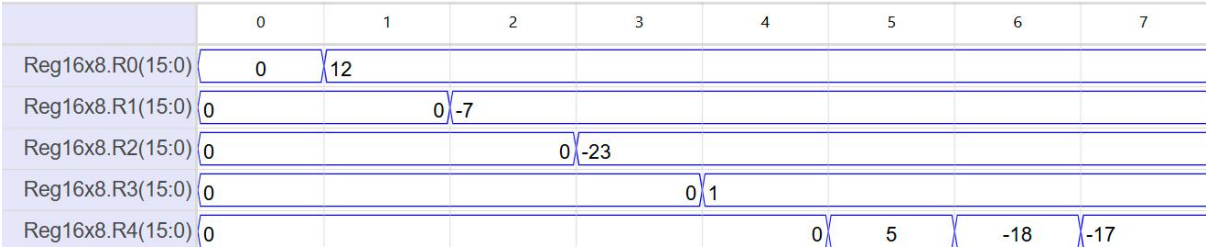
Simulation is on the next page.



2: Explain why having only a single read port from the register file to the ALU would severely limit the CPU's ability to perform useful arithmetic operations in a single cycle.

Instructions such as ADD R3, R0, R1 require both source operands to be present at the ALU simultaneously. A single read port would allow only one operand to be fetched per cycle, forcing multi-cycle execution or the use of temporary storage — reducing throughput and increasing hardware complexity. Two independent read ports allow both operands to reach the ALU in a single cycle.

3: Write a new text file containing the four lines of assembly code in Figure 6. Add three more lines of code that add the contents of registers R0–R3 and store the result in R4, without modifying registers R5–R7. Simulate your code in eep1lab1 and use the waveform viewer to confirm that the result in R4 is correct.

```
MOV R0, #12
MOV R1, #-7
MOV R2, #233
MOV R3, #1
ADD R4, R0, R1
ADD R4, R4, R2
ADD R4, R4, R3
```

Expected result: 12 + (−7) + 233 + 1 = 239 (0x00EF). The waveform confirms R4 = 239 after cycle 7, with R5–R7 unchanged throughout.